Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Kierunek: **Informatyka Algorytmiczna**

PRACA DYPLOMOWA
INŻYNIERSKA

Zabezpieczanie modeli językowych za pomocą cyfrowych znaków wodnych

Aleksandra Wójcik

Opiekun pracy
Dr inż. Marcin Zawada

Słowa kluczowe: Model językowy, Watermark, NanoGPT

WROCŁAW 2024

STRESZCZENIE

Niniejsza praca została stworzona z myślą o implementacji algorytmów zabezpieczających modele językowe cyfrowym znakiem wodnym. Głównym zamysłem było również wykonanie i dogłębne opisanie badań wpływu przedstawionego zabezpieczenia i dobranych parametrów na wyniki generowane przez model.

Praca składa się z trzech głównych części - teoretycznej, implementacyjnej oraz badawczej. Pierwsza z nich szczegółowo opisuje zagadnienia niezbędne do zrozumienia działania modeli językowych. Przedstawiona zostaje architektura transformer, będąca bazą większości modeli przetwarzających język naturalny. Opisywane są także tematy związane z tokenizacją, wektorami osadzeń oraz samoatencją. Poruszone zostały kwestie trenowania i dostrajania modeli oraz przeanalizowano proces generowania tekstu. Część teoretyczna posiada również opis metod watermarkingu wraz z ich procesem detekcji.

Kolejno uwaga skupia się na praktycznym zastosowaniu opisanych wcześniej metod. Wszelkie implementacje zostały przeprowadzone na stworzonym przez *Andrej'a Karpathy'ego* minimalistycznym modelu *NanoGPT*. Omówione zostają także zagadnienia związane ze złożonością obliczeniową.

Finalnie praca koncentruje się na aspekcie badawczym. Przedstawione zostają wyniki badań dla różnych parametrów obieranych w algorytmach watermarkingu pod kątem mocy zabezpieczenia oraz uzyskiwanej jakości generowanego tekstu. Otrzymywane wyniki są wizualizowane w formie wykresów oraz poddawane analizie matematycznej. Pracę zamykają wnioski sformułowane w oparciu o przedstawioną analizę i wyniki.

ABSTRACT

This thesis was created with the aim of implementing algorithms that secure language models through watermarking. The main intention was to execute and describe researches of the impact of used algorithms and chosen parameters on outcome generated by model.

This paper contains three main sections - theoretical, implementation and research. The first section described essential topics for understanding the operation of language models. The Transformer architecture forming the basis of most models is introduced. This study describes also topics related to tokenization, embedding vectors and self-attention.

Processes of training, fine-tuning, and text generation have been discussed. The theoretical part also contains informations about watermark algorithms and detection process.

Next, this paper focuses on the practical application of described methods. All implementations were executed on minimalistic NanoGPT created by *Andrej Karpathy*. Certain issues on computational complexity have also been discussed.

Lastly, the thesis focuses on the research aspect. The results of reaserches of the impact of choosen parameters used is watermarking algorithms are presented. Recieved results are visualized in charts and are subjected to mathematical analysis. Conclusions are formulated based on the findings of the study.

SPIS TREŚCI

Wprowadznie 4

Analiza problemu 4

Cel pracy 4

Struktura pracy 4

1. Opis modeli językowych 6

1.1. Wprowadzenie do modeli językowych 6

1.2. Architektura Transformer 6

1.3. Koder i dekoder 6

1.3.1. Koder 7

1.3.2. Dekoder 8

1.4. Tokenizacja 8

1.5. Osadzenia tokenów 9

1.6. Osadzenia pozycyjne 10

1.7. Mechanizm samoatencji 11

1.7.1. Macierze zapytań, kluczy i wartości 11

1.7.2. Obliczanie samoatencji 12

1.8. Głowy atencji 13

1.9. Warstwy propagacji w przód i normalizacja warstw 14

1.9.1. Warstwa propagacji 14

1.9.2. Normalizacja warstw 15

2. Generowanie tekstu 16

2.1. Przewidywanie następnego słowa 16

2.1.1. Autoregresywny proces wyboru tokenów 16

2.1.2. Ustalanie prawdopodobieństw dla tokenów 16

2.2. Strategie wyboru następnych tokenów 16

2.2.1. Greedy Decoding 16

2.2.2. Beam search decoding 17

2.2.3. Random Sampling 17

2.2.4. Top-k Sampling 17

2.2.5. Temperatura 18

3. Trenowanie i dostrajanie modeli językowych 19

3.1. Wprowadzenie do trenowania modeli językowych 19

3.2. Pre-training 20

3.3. Fine-Tuning . . . 20
3.4. Metryka perplexity . . . 20
3.5. Punkty kontrolne . . . 20

4. NanoGPT . . . 21

4.1. Opis NanoGPT . . . 21
4.2. Architektura NanoGPT . . . 21

5. Zabezpieczanie modeli językowych . . . 23

5.1. Korzyści związane z zabezpieczaniem modeli językowych . . . 23
5.2. Prosta koncepcja zabezpieczenia . . . 23
5.3. Problemy związane z zabezpieczaniem modeli językowych . . . 23
5.4. Silny znak wodny . . . 24
5.4.1. Koncepcja . . . 24
5.4.2. Algorytm silnej czerwonej listy . . . 24
5.5. Lekki znak wodny . . . 25
5.5.1. Koncepcja . . . 25
5.5.2. Algorytm lekkiej czerwonej listy . . . 25
5.6. Detekcja cyfrowego znaku wodnego . . . 26
5.7. Problemy związane z detekcją lekkiego znaku wodnego . . . 27
5.7.1. Spike entropy . . . 27
5.7.2. Wartość oczekiwana rozmiaru zielonej listy . . . 28
5.7.3. Wnioski . . . 28
5.8. Wpływ cyfrowego znaku wodnego na jakość generowanego tekstu . . . 28

6. Rozwój i dostrajanie modelu NanoGPT . . . 29

6.1. Wprowadzenie . . . 29
6.2. Silny znak wodny . . . 29
6.3. Lekki znak wodny . . . 31
6.4. Detekcja cyfrowego znaku wodnego . . . 31
6.5. Złożoność obliczeniowa . . . 32
6.6. Dostrajanie modelu . . . 33
6.6.1. Transfer wag wytrenowanego modelu . . . 33
6.6.2. Dostrajanie NanoGPT . . . 33

7. Badania i analiza wyników . . . 34

7.1. Przeprowadzone badania . . . 34
7.1.1. Badanie wpływu wielkości zielonej listy na moc znaku wodnego i jakość tekstu . . . 34
7.1.2. Aspekt matematyczny . . . 35
7.1.3. Badanie wpływu parametru mocy na moc znaku wodnego i jakość tekstu 35
7.1.4. Aspekt matematyczny . . . 36
7.1.5. Jednoczesne badanie parametru γ i δ . . . 37

7.1.6. Badania algorytmu silnej czerwonej listy 37
7.1.7. Aspekt matematyczny 38
7.2. Analiza wyników 39

Podsumowanie 40

Omówienie zawartości pracy 40
Efektywność przestawionych algorytmów 40
Wyzwania związane z implementacją 40

Bibliografia 41

Spis rysunków 43

Spis tabel 44

WPROWADZNIE

ANALIZA PROBLEMU

Rosnąca popularność narzędzi przetwarzania języka naturalnego, takich jak ChatGPT od OpenAI otworzyła wiele nowych możliwości w zakresie generowanie treści, ale także znacząco przyczyniła się do wzrostu liczby przypadków plagiatów. Omawiany mechanizm jest w stanie wytwarzać wysokiej jakości tekst, bazując na ogromnej ilości dostępnych informacji. Przyczyniło się to do wzrostu ilości przypadków, w których użytkownicy bez modyfikacji kopiują wygenerowane treści i publikują je jako własne. Problem ten jest szczególnie zauważalny w obszarze edukacji oraz publikacji naukowych, gdzie autentyczność odgrywa kluczową rolę.
Rozwój technologii przetwarzania języka naturalnego przyczynił się także do szerzenia nieprawdziwych treści w internecie. Treści generowane przez modele językowe mogą powodować również szerzenie się dezinformacji we wszystkich mediach.
W związku z powyższym, konieczne jest stosowanie strategii zabezpieczania modeli, które umożliwiają wykrycie generowanych przez nie treści. Jednym z takich rozwiązań są cyfrowe znaki wodne, zwane również jako watermarki. Wdrożenie tej technologii w modelach językowych umożliwia identyfikacje źródła pochodzenia tekstu.

CEL PRACY

Niniejsza praca powstała w celu zaimplementowania zabezpieczeń modelu językowego *NanoGPT* za pomocą cyfrowego znaku wodnego oraz wdrożenia algorytmów detekcji watermarków w przetwarzanych tekstach. Ponadto model został poddany *fine-tuningowi* na charakterystycznych danych. Dodatkowo w pracy zostały opublikowane rezultaty badań, opisujących wpływ różnych parametrów algorytmu na jakość generowanego tekstu.

STRUKTURA PRACY

Praca została podzielona na siedem rozdziałów. Pierwsze pięć rozdziałów zostało napisanych w celu opisania zagadnień teoretycznych, natomiast pozostałe skupiły się na szczegółowym opisie implementacji oraz wnioskach z przeprowadzonych badań. W rozdziale 1 zawarto szczegółowy opis modeli przetwarzania języka naturalnego, wprowadzono terminologie umożliwiającą zrozumienie dalszej części pracy oraz objaśniona została architektura Transformer. Rozdział 2 koncentrował się na procesie generowania tekstu,

czyli istocie działania modeli językowych, natomiast w rozdziale 3 został opisany przebieg trenowania i dostrajania modeli językowych. W rozdziale 5 przedstawiony został nanoGPT, na którym zaimplementowane zostały omawiane w rozdziale 6 algorytmy. Wyniki przeprowadzonych badań zostały przedstawione i szeroko omówione w rozdziale 7.

1. OPIS MODELI JĘZYKOWYCH

1.1. WPROWADZENIE DO MODELI JĘZYKOWYCH

Modele językowe zyskują coraz większą sławę, ze względu na szeroki wachlarz zastosowań. Modele są używane w wielu dziedzinach, takich jak tłumaczenie i klasyfikacja tekstów. Natomiast przede wszystkim znane są jako narzędzia do generowanie tekstów o interdyscyplinarnej tematyce.

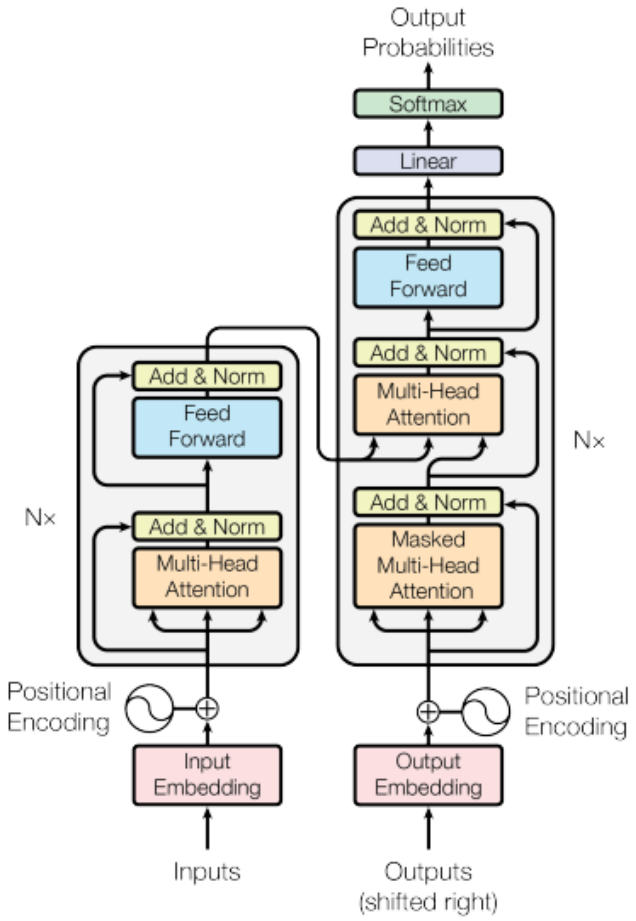
1.2. ARCHITEKTURA TRANSFORMER

Ten rozdział został poświęcony przedstawieniu architektury Transformer. Omawiana architektura została przedstawiona w pracy naukowej *Attention is All You Need* [15] opublikowanej w 2017 roku.
Transformer wykorzystuje mechanizm atencji oraz przetwarza wiele sekwencji równolegle, przez co obliczenia przeprowadzane przez model w trakcie treningu lub generacji są szybsze w porównaniu do wcześniej wykorzystywanych podejść. Ponadto ta architektura umożliwia uchwycenie całego kontekstu przetwarzanego tokenu. Ze względu na swoje zalety, architektura stała się najczęściej stosowaną architekturą wśród modeli językowych.

1.3. KODER I DEKODER

Architektura transformera zakłada istnienie dwóch podstawowych komponentów - kodera i dekodera. Mogą one współpracować ze sobą tworząc model typu koder-dekoder np. *Bidirectional and auto-regressive transformers* - BART [25]. Możliwe jest również rozdzielenie komponentów tworząc w ten sposób modele typu *coder-only* np. *Bidirectional encoder representations from transformers* - BERT [14] lub *decoder-only* np. *Generative pre-trained transformer* - GPT [34]. Nie są to jednak wszystkie dostępne architektury. Aktualnie biblioteka *Transformers*, należąca do platformy *Hugging Face* [3] oferującej narzędzia umożliwiające tworzenie zaawansowanych modeli językowych, oferuje ponad 50 różnych architektur.
Czym zatem różni się koder i dekoder? W celu dogłębnego wyjaśnienia omawianej różnicy, należy przeanalizować architekturę transformera z rys. 1.1 opublikowaną w artykule

naukowym "Attention is All You Need". Po lewej umiejscowiony jest koder, natomiast po prawej - dekoder.



Rysunek 1.1: Architektura koder dekoder [15]

1.3.1. Koder

Celem kodera jest przekształcenie tekstu wejściowego w macierz reprezentującą znaczenie danych w wielowymiarowej przestrzeni. Proces ten rozpoczyna się od tokenizacji tekstu omówionej w podrozdziale 1.4, następnie dla uzyskanych tokenów inicjowane są wektory osadzeń scharakteryzowane w sekcji 1.5, które są przekazywane do stosu kodera (zaznaczonego kolorem szarym na rys. 1.1).

Danymi wejściowymi dla stosu kodera (zaznaczonego kolorem szarym na rys. 1.1) są wektory osadzeń dla tokenów otrzymanych w procesie tokenizacji przetwarzanej sekwencji. Opisywany stos kodera składa się z wielu warstw ułożonych jedna za drugą nazywanymi blokami transformera. W skład każdego z nich wchodzi wielogłowowa warstwa samoatencji przedstawione w sekcjach 1.7 i 1.8 oraz warstwa propagacji w przód opisana w podrozdziale 1.9.1. Ponadto architektura kodera wykorzystuje normalizację warstw ujętą w sekcji 1.9.2.

Efektem pracy kodera są otrzymywane *reprezentacje kontekstowe*, które zawierają informacje o znaczeniu każdego tokena w kontekście przetwarzanej sekwencji.

1.3.2. Dekoder

Głównym zadaniem dekodera jest wyznaczenie następnego tokenu bazując na przetwarzanej sekwencji.

Wejściem do stosu dekodera są wektory osadzeń tokenów opisane w sekcji 1.5. Ponadto dekoder może także być dostosowany do przyjmowania wyjścia kodera.

Opisywany stos dekodera składa się z wielu bloków transformera. Bloki transformera występujące w dekoderze mają identyczną strukturę jak te wykorzystywane w koderze, jednakże w tym przypadku używana jest warstwa maskowanej samoatencji przedstawiona w podrozdziale 1.7. (Dla architektury typu np *koder - dekoder* wykorzystywana jest dodatkowa warstwa samoatencji w blokach transformera w stosie dekodera)

W wyniku przepuszczenia osadzeń przez stos dekodera, dekoder wytwarza reprezentacje kontekstowe tokenów, które następnie są one poddawane przekształceniu liniowemu. Wynikiem takiej operacji są tzw. *logity*. Każdy logit odpowiada dokładnie jednemu tokenowi i przyjmuje wartości należące do $\mathbb{R}$. Odpowiadają za odzwierciedlenie preferencji modelu względem każdego tokenu ze słownika $\mathcal{V}$. Wraz ze wzrostem wartości logitu wzrasta również zainteresowanie modelu względem tokenu, któremu ten logit odpowiada.

Następnie logity są poddawane normalizacji przy użyciu funkcji *softmax* wyrażającej się wzorem:

$$\sigma(\mathbf{l})_i = \frac{e^{l_i}}{\sum_{j=1}^{|\mathcal{V}|} e^{l_j}} \quad \text{gdzie } l_i \text{ to i-ty logit}\,. \tag{1.1}$$

Użycie tej funkcji skutkuje wytworzeniem się prawdopodobieństw używanych do predykcji następnego tokenu w sekwencji.

1.4. TOKENIZACJA

Modele nie przyjmują bezpośrednio na wejściu zwyczajnych sekwencji tekstu. Należy w odpowiedni sposób przygotować dane, na których operuje model. Z tej przyczyny sekwencje wejściowe są poddawane tokenizacji. Tokenizacja to etap dzielenia sekwencji na mniejsze, niepodzielne części, zwane tokenami. Zbiór wszystkich tokenów występujących w modelu, gdzie każdy z nich ma swój unikalny **identyfikator** nazywany jest *słownikiem* i oznaczany jest jako $\mathcal{V}$. Tokeny mogą być pojedynczymi znakami, ciągami znaków, a także słowami. Poniżej przedstawiona tabela prezentuje najczęściej stosowane techniki tokenizacji.

Zdecydowana większość modeli językowych stosuje ostatnią z przedstawionych metod, czyli tokenizację podwyrazową. Omawiana technika góruje nad tokenizacją znakową, ze względu na bardziej skuteczne uchwycenie kontekstu oraz minimalizacje problemu związanego z generowaniem nieistniejących wyrazów. Natomiast tokenizacja wyrazowa

Metoda tokenizacji	Proces
Tokenizacja znakowa	Każdy znak traktowany jest jako osobny token.
Tokenizacja wyrazowa	Każde słowo traktowane jest jako osobny token.
Tokenizacja podwyrazowa	Tekst jest dzielony na mniejsze jednostki o nieokreślonej długości. W tym procesie stosuje się algorytm unigram.

Tabela 1.1: Metody tokenizacji

wymaga większej ilości pamięci do przechowywania wszystkich tokenów. Ze względu na powyżej opisane mankamenty pozostałych metod, tokenizacje na poziomie podciągów uznaje się jako najbardziej optymalną.

W celu dokładniejszego zrozumienia procesu tokenizacji, poniżej zaprezentowano przykład, w jaki sposób tokenizator wykorzystywany przez OpenAI w GPT-3.5/GPT-4 dokonuje podziału zdania *Zabezpieczanie modeli językowych za pomocą cyfrowych znaków wodnych*

Rysunek 1.2: Procesu tokenizacji dla modeli GPT-3.5/GPT-4

Zdanie zostało przekształcone na 25 różnych tokenów, a rozbicie to zostało zaznaczone na grafice 1.2 różnymi kolorami.
Końcowym etapem tokenizacji jest przypisanie tokenom unikalnej wartości liczbowej, będącej ich **identyfikatorem**. Dla wcześniej omawianego wyrazu, w rezultacie otrzymujemy tablicę z 25 unikalnymi elementami.
Prezentuje się ona następująco:
$[57, 84578, 20898, 14088, 19700, 1646, 72, 503, 5267, 4341, 74, 47011, 15036,$
$95811, 5985, 9693, 69, 654, 5759, 30974, 587, 12543, 289, 347, 28383]$

1.5. OSADZENIA TOKENÓW

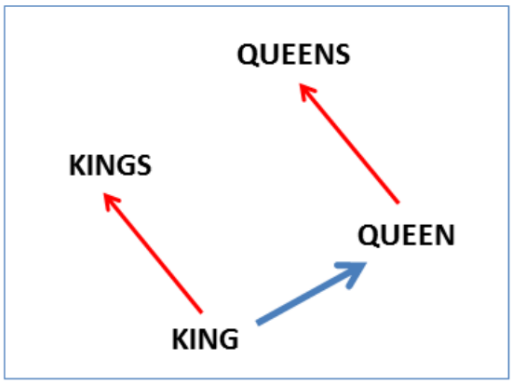
Procesem następującym po tokenizacji, również mającym na celu odpowiednie przygotowanie danych do przetwarzania przez model jest uzyskanie wektorów osadzeń dla tokenów.

Osadzenia to reprezentacje tokenów w przestrzeni wektorowej o wymiarze d, każdemu tokenowi t_i przypisany jest wektor osadzenia $\overrightarrow{\mathbf{E}}_i \in \mathbb{R}^d$ Wektory reprezentują zadane ciągi znaków i zależności pomiędzy nimi. Zakładając, że tokeny reprezentuje wektor o kilkuset wymiarach, każdy z elementów wektora jest pewną wagą wpływającą na jego znaczenie.

Rys. 1.3 ukazuje hipotetyczne wagi i wymiary w wektorze. W ten sposób osadzenie nie tylko jest w stanie reprezentować token, ale także relacje między nimi.

Rysunek 1.3: Hipotetyczne przedstawienie wektorów osadzeń [12].

Opisywana właściwość prowadzi do wystąpienia stałych przesunięć wektorów między tokenami w relacjach, takich jak płeć, liczba pojedyncza i mnoga. W celu zobrazowania działania mechanizmu, po raz kolejny zostanie przeanalizowana grafika Adriana Colyer'a 1.4:



Rysunek 1.4: Relacja zachodzące między wektorami [12].

Grafika 1.4 ukazuje działanie na wektorach *Król - Mężczyzna + Kobieta* = ***Królowa***. Przedstawiony przykład ilustruje w jaki sposób możliwe jest manipulowanie zależnościami mającymi wpływ na wektory.

1.6. OSADZENIA POZYCYJNE

Modele bazujące na architekturze transformer przetwarzają całe sekwencje równocześnie. W związku z tym istnieje problem z rozróżnieniem pozycji poszczególnych tokenów. Rozwiązaniem tego kłopotu jest wprowadzenie mechanizmu osadzeń pozycyjnych, które pozwalają na gromadzenie informacji dotyczących kolejności występowania tokenów w sekwencji. Finalnie osadzenia tokenów łączy się z osadzeniami pozycyjnymi przed przeka-

zaniem ich do stosu kodera lub dekodera.

Zazwyczaj osadzenia pozycyjne traktuje się jako parametry, których model uczy się w trakcie treningu. Działanie jest adekwatne do trenowania osadzeń tokenów, jednak zamiast identyfikatorów tokenów, stosuje się indeksy pozycyjne.

Alternatywą przedstawioną w artykule *Attention Is All You Need* [10] jest używanie osadzeń pozycyjnych sinusoidalno-cosinusoidalnych . Dla każdej pozycji w sekwencji generowane są różne wartości w zależności od zadanego poniżej wzoru:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right), \tag{1.2}$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right), \tag{1.3}$$

gdzie

— pos to pozycja tokenu,
— i to indeks wymiaru osadzeń,
— d_{model} to wymiar wektorów osadzeń,

1.7. MECHANIZM SAMOATENCJI

Samoatencja umożliwia modelowi zrozumienie relacji występujących pomiędzy tokenami w przetwarzanych tekstach. Przetwarzany tekst jest w postaci macierzy zawierającej wektory osadzeń dla użytych tokenów.

1.7.1. Macierze zapytań, kluczy i wartości

Mechanizm samoatencji wykorzystuje macierze danych zapytań, kluczy i wartości

— **Macierz zapytań** Q określa informacje, które model chce wyszukać w zbiorze danych.
— **Macierz kluczy** K to macierz reprezentująca cechy tokenów.
— **Macierz wartości** V to macierz zawierająca dane, które mają być przekazane do dalszej analizy.

Przekształcenie liniowe wektorów osadzeń $\overrightarrow{\mathbf{E}}_i$ tworzy tworzy wektory Q_i, K_i i V_i, które są wierszami opisanych wcześniej macierzy Q, K i V. Ten zabieg jest wykonywany poprzez przemnożenie wektorów osadzeń $\overrightarrow{\mathbf{E}}_i$ przez odpowiednie macierze wag W_Q, W_K i W_V

1.7.2. Obliczanie samoatencji

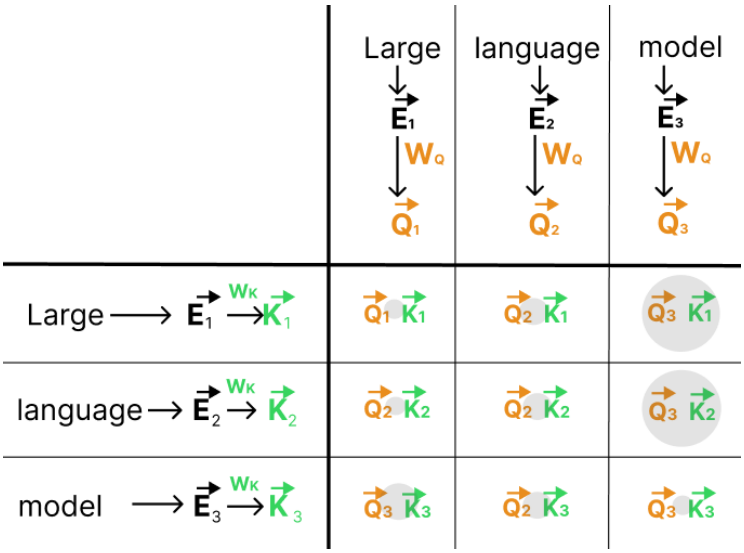
Atencja jest wyliczana według następującego wzoru

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V. \tag{1.4}$$

Zatem pierwszym etapem wyliczania atencji jest obliczenie iloczynu skalarnego dla wektorów

Dla każdej pozycji i w sekwencji o długości T obliczamy podobieństwo między zapytaniem Q_i a każdym kluczem K_j, gdzie $j \in [1, T]$ z pomocą iloczynu skalarnego. Wyniki obliczeń umieszczane są w macierzy wynikowej M. Wysokie wartości iloczynów skalarnych, świadczą o dużej korelacji pomiędzy wektorem zapytania a wektorem klucza.

Przykładowy rezultat przedstawiony został na rys. 1.5, wielkość kół jest proporcjonalne do wielkości iloczynu skalarnego.



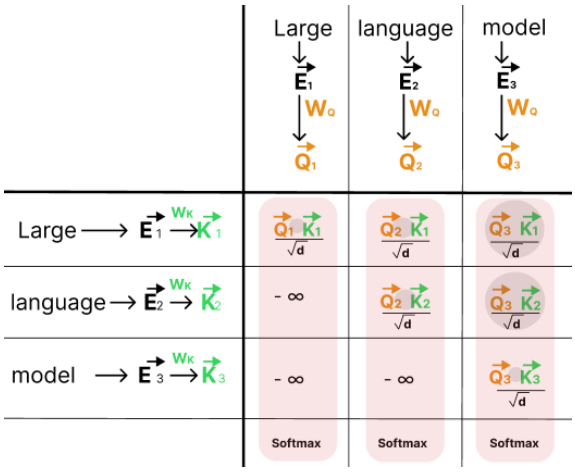
Rysunek 1.5: Przykładowa macierz $M = Q \cdot K$

W przypadku dekodera, gdzie stosowana jest warstwa maskowanej samoatencji, dodawana jest maska, która pozwala modelowi skupić się wyłącznie na przetwarzaniu tokenów z wcześniejszych pozycji. Zatem dla wszystkich wektorów Q_i i K_j, gdzie $i, j \in [1, T]$

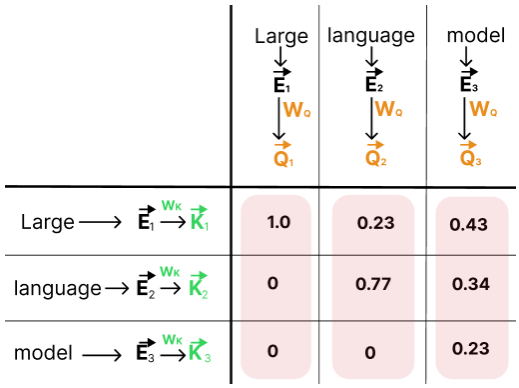
$$\forall_{i,j\,:\,i<j}\; M_{j,i} = -\infty.$$

Ponadto każde pole macierzy dzielone jest przez $\sqrt{d}$, a następnie każdą kolumnę macierzy poddaje się normalizacji poprzez zastosowanie funkcji softmax, w celu uzyskania prawdopodobieństw.

Finalnie ponownie wylicza się iloczyn skalarny dla każdego wektora wartości V_i i wcześniej uzyskanych wektorów prawdopodobieństw. W rezultacie dla każdego przetwarzanego



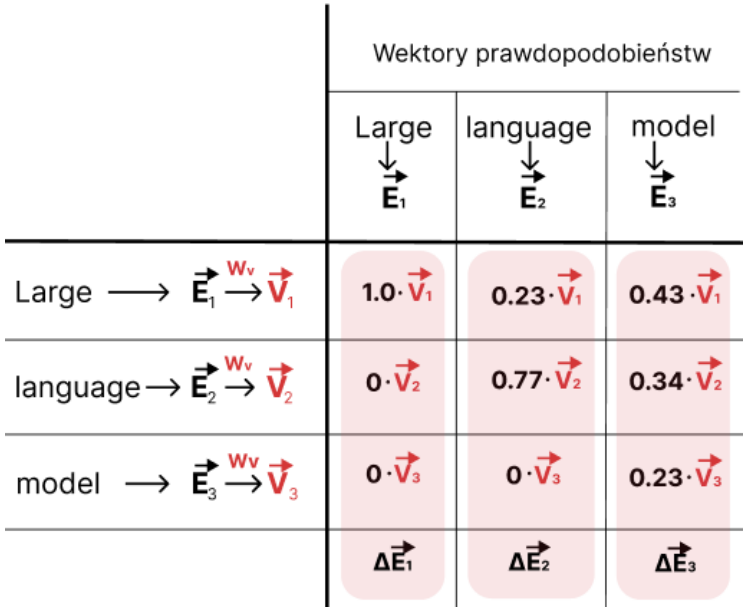
Rysunek 1.6: Proces maskowania.



Rysunek 1.7: Macierz przykładowych prawdopodobieństw pomiędzy wektorami zapytań i kluczy.

osadzenia $\overrightarrow{\mathbf{E}}_i$, uzyskuje się $\Delta\overrightarrow{\mathbf{E}}_i$, która posłuży do zaktualizowania wektora osadzenia według wzoru

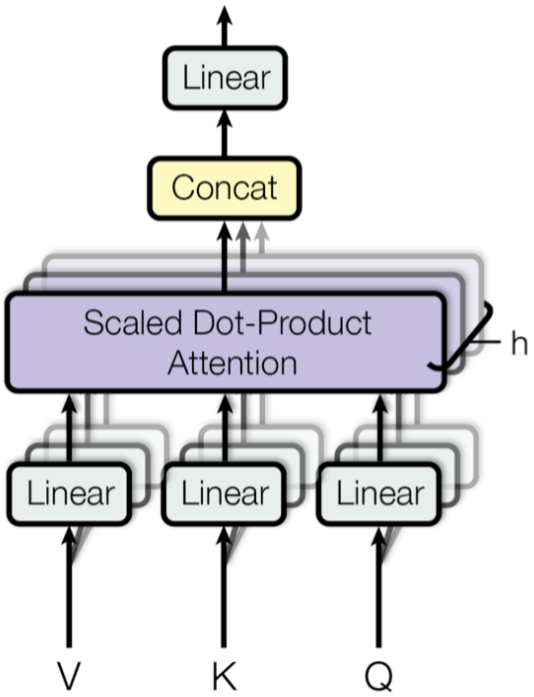
$$\overrightarrow{\mathbf{E}'}_i = \overrightarrow{\mathbf{E}}_i + \Delta\overrightarrow{\mathbf{E}}_i. \tag{1.5}$$



Rysunek 1.8: Proces tworzenia $\overrightarrow{\mathbf{E}}_i$ dla przykładowych prawdopodobieństw.

1.8. GŁOWY ATENCJI

Warstwa samoatencji stosuje wykorzystuje niezależne transformacje liniowe, które są odpowiedzialne za wytworzenie macierzy Q, K i V. Zdecydowaną poprawę uzyskiwanych wyników zapewnia zastosowanie wielu zestawów tych projekcji liniowych - to podejście nazywane jest atencją wielogłowową. Ten rodzaj atencji obrazuje rys. 1.9.



Rysunek 1.9: Wiele głów atencji [10].

Każda z głów jest w stanie analizować przetwarzaną sekwencje pod różnym kątem podobieństw. Przyczynia się to do zwiększenia umiejętności modelu w wychwytywaniu bardziej złożonych zależności w sekwencjach. Poniżej przedstawiono wzór opublikowany został w pracy *Attention is all you need* [10].

$$\text{MultiHead}\left(\mathbf{Q}, \mathbf{K}, \mathbf{V}\right) = \left[\text{head}_1, \ldots, \text{head}_h\right] \mathbf{W}_0, \tag{1.6}$$

$$\text{gdzie } \text{ head}_i = \text{Attention}\left(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V\right), \tag{1.7}$$

gdzie wszystkie $\mathbf{W}_i$ to macierze, których wagi podlegają procesowi uczenia.

1.9. WARSTWY PROPAGACJI W PRZÓD I NORMALIZACJA WARSTW

1.9.1. Warstwa propagacji

W skład każdego bloku transformera wchodzi warstwa propagacji w przód, w której każdy wektor osadzeń przetwarzany jest niezależnie. Opisywana warstwa jest odpowiedzialna za zapamiętywanie istotnych informacji, które są przekazywane do modelu jako dane treningowe. Proces zapamiętywania jest możliwy ze względu na możliwość ustalenia wag neuronów występujących w warstwie podczas procesu uczenia.

1.9.2. Normalizacja warstw

Normalizacja warstw to technika mająca na celu stabilizacje i poprawę wydajności procesu uczenia. Konieczność stosowania omawianego mechanizmu wynika z występowania problemu *wewnętrznego przesunięcia kowariancji*, który jest przyczyną występowania zmian w rozkładzie aktywacji neuronów w warstwie podczas procesu uczenia. Zatem normalizacje warstw stosuje się w celu wyrównania średniej do 0 oraz wariancji do 1 dla każdej danej wejściowej występującej w sekwencji.

2. GENEROWANIE TEKSTU

2.1. PRZEWIDYWANIE NASTĘPNEGO SŁOWA

2.1.1. Autoregresywny proces wyboru tokenów

Autoregresywny proces wyboru tokenów to technika stosowana podczas generowania tekstu. Polega na przewidywaniu następujących tokenów krok po krok, bazując na wcześniejszych wygenerowanych tokenach. Proces ten powtarza iteracyjnie i zakańcza się w momencie wygenerowania tokenu specjalnego lub wytworzenia ustalonej, maksymalnej liczby tokenów.

2.1.2. Ustalanie prawdopodobieństw dla tokenów

Podczas generowania tekstu, model analizuje całą sekwencję przetwarzanego tekstu. Następnie, dla każdego tokenu występującego w sekwencji, stos dekodera wytwarza reprezentacje kontekstowe. Każdy reprezentacja jest poddawana transformacji liniowej, wskutek czego otrzymywane są logity, które poddawane są funkcji softmax, w celu uzyskania prawdopodobieństwa. Otrzymywane jest prawdopodobieństwo $P(y|x)$ gdzie y=y_1,y_2,...y_t jest sekwencja wygenerowanych tokenów na podstawie pewnej zadanej sekwencji x=x_1,x_2,...,x_k, co jest jednoznaczne z zapisem

$$P(y_1, y_2, ...y_t|x) = \prod_{t=1}^{n} P(y_t|y_{<t}, x). \tag{2.1}$$

Kolejno na podstawie otrzymanych prawdopodobieństw model dokonuje predykcji następnego tokenu.

2.2. STRATEGIE WYBORU NASTĘPNYCH TOKENÓW

2.2.1. Greedy Decoding

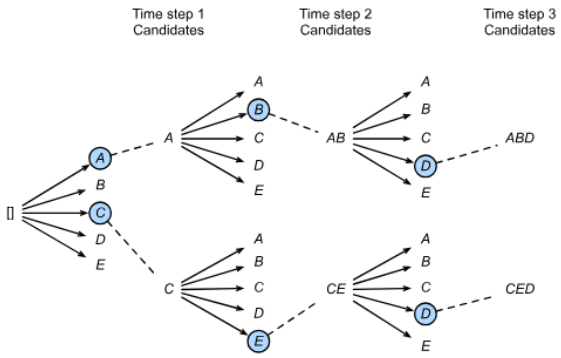
Najprostszą metodą wyboru następnego w sekwencji tokenu jest *Greedy Decoding*. Polega ona na wyborze tokena, którego prawdopodobieństwo w danym kroku jest najwyższe.

$$y_t = argmax_{y_t} P(y_t|y_{<t}, X). \tag{2.2}$$

Główną zaletą takiego podejścia jest szybkość działania, niestety ze względu na deterministyczny charakter *greedy encoding* istnieje problem związany z brakiem różnorodności generowanych tekstów. Każdorazowe wybieranie najbardziej prawdopodobnego tokenu w kontekście całej sekwencji może skutkować niepoprawnymi wynikami. Wynika to z braku rozważania dalszych alternatyw.

2.2.2. Beam search decoding

W przeciwieństwie do pierwszej metody, *Beam search decoding* śledzi zbiór b najbardziej prawdopodobnych tokenów, gdzie b nazywana jest liczbą hipotez częściowych. Każdorazowo omawiany zbiór, zostaje ustalany poprzez przedłużenie każdego z elementów o jeden token i ponowny wybór b najbardziej prawdopodobnych tokenów. Przestawiony poniżej rys. 2.1 trafnie przedstawia opisywany schemat działania.



Rysunek 2.1: Beam Search [21].

2.2.3. Random Sampling

Random Sampling do często stosowana metoda próbkowania, polega na losowym wyborze tokena na podstawie rozkładu prawdopodobieństw wyników nad całym słownikiem $\mathcal{V}$ w każdym kroku iteracji.

$$P(y_t = w_i | y_{<t}, x) = softmax(z_{t,i}) = \frac{exp(z_{t,i})}{\sum_{j=1}^{|V|} z_{t,j}}. \tag{2.3}$$

Stosowanie *Random Sampling* pomaga w generowaniu bardziej różnorodnych wyników. Jednak, losowość może przyczynić się do uzyskania niepoprawnych tekstów. W celu ograniczenia takiej możliwości użyteczna jest kolejno opisana technika próbkowania - Top-K Sampling.

2.2.4. Top-k Sampling

Ta technika jest identyczna do wcześniej omawianej *Random Sampling*, jednak zakłada ona próbkowania jedynie z wyznaczonego k najbardziej prawdopodobnych tokenów. W

ten sposób zapewniona jest różnorodność generowanych wyników, a jednocześnie prawdopodobieństwo wybrania niepoprawnego tokenu jest minimalizowane.

2.2.5. Temperatura

Parametr temperatury $\sqcup$ odpowiada za sterowanie stopniem losowości wygenerowanych wyników. Pozwala on na regulowanie rozkładu prawdopodobieństwa tokenów, poprzez przeskalowanie logitów przed obliczeniem funkcji softmax, w następujący sposób:

$$P(y_t = w_i | y_{<t}, x) = softmax(z_{t,i}) = \frac{exp(z_{\sqcup,i}/\sqcup)}{\sum_{j=1}^{|V|} (z_{\sqcup,j}/\sqcup)}. \tag{2.4}$$

Dla $\sqcup < 1$ rozkład prawdopodobieństwa jest bardziej spiczasty, co powoduje tłumienie mniej prawdopodobnych tokenów. Natomiast w przeciwnej sytuacji rozkład jest spłaszczony, co skutkuje wystąpieniem mniejszych różnic w prawdopodobieństwach występujących pomiędzy tokenami.

3. TRENOWANIE I DOSTRAJANIE MODELI JĘZYKOWYCH

3.1. WPROWADZENIE DO TRENOWANIA MODELI JĘZYKOWYCH

Uczenie modeli językowych rozumienia języka naturalnego jest uzyskiwane podczas procesu trenowania. Efekt ten jest osiągany, poprzez przekazanie do modelu ogromnej ilości danych treningowych, będącymi tekstami różnego pochodzenia. Treści mogą pochodzić z różnych źródeł, takich jak książki czy strony internetowe.

Jednakże, niezbędne jest przeprowadzenie *pre-processing'u* na zgromadzonych zbiorach danych zanim trafią one do modelu. W ramach tego procesu wykorzystuje się różnorodne techniki, w tym usuwanie błędnych lub zduplikowanych danych.

Przygotowane dane są przekazywane do modelu, gdzie odbywa się uczenie nadzorowane. Dane są dzielone w pary danych wejściowych oraz odpowiadającym im etykiet. Dla zadanych danych wejściowych wprowadzonych do modelu, powinien on być w stanie przewidzieć kolejne tokeny dopowiadające etykietom. W tym celu stosuje się funkcje straty, która odpowiada za obliczenie różnicy między predykcjami modelu a rzeczywistymi etykietami.

Modele językowe w tym celu wykorzystują entropia krzyżowa, która jest odpowiedzialna za sprawdzenie czy model przypisuje tokenom zgodnym z etykietami odpowiednio duże prawdopodobieństwa. Krzyżowa entropia wyliczana jest w następujący sposób:

$$\text{Entropia krzyżowa} = -\frac{1}{S}\sum_{i}\log(p_{t_i}), \tag{3.1}$$

gdzie:

— S - długość sekwencji,
— i - pozycja w sekwencji,
— t_i - indeks prawdziwej etykiety tokenu na pozycji i,
— p_{t_i} - prawdopodobieństwo prawdziwej etykiety tokenu na pozycji i.

Ze względu na dobrze zdefiniowane pochodne, krzyżowa entropia jest często używana do trenowania modeli przy użyciu optymalizacji gradientowej, ponieważ uzyskany wektor gradientów skutecznie wskazuje kierunek, w którym należy aktualizować wagi. Celem treningu jest dobranie takich wag modelu, które minimalizują funkcje straty.

3.2. PRE-TRAINING

Pre-training to początkowa faza trenowania modeli językowych, podczas której modele uczone są rozumienia ogólnych kontekstów i zależności występujących w danych treningowych. Podczas tego etapu model przyswaja wiedze z ogromnej ilości danych, mogą być to książki, artykuły i wiele więcej. W efekcie model nie jest dostosowany do żadnego konkretnego zadania, zatem po zakończeniu tej fazy, zostaje on poddany *fine-tuningowi*. O modelu który odbył *pre-training* mówi się, że jest on wstępnie wytrenowany.

3.3. FINE-TUNING

Aby dostosować wstępnie wytrenowany model do określonego typu zadań, należy go dostroić. Taką funkcję pełni *fine-tuning*. W wyniku *fine-tuningu* parametry wstępnie wytrenowanego modelu, są ponownie poddawane treningowi, jednak na zdecydowanie mniejszych, specyficznych zbiorach danych, dostosowanych do określonego typu zadania. W ten sposób modelu uzyskuje umiejętność generowania specyficznych treści w kontekście zadanego zagadnienia.

3.4. METRYKA PERPLEXITY

Perplexity jest metryką służącą do oceny jakości modeli językowych. Dla danego modelu i zadanej sekwencji, perplexity ustala jak bardzo prawdopodobne jest jej wygenerowane. Dla zadanej tokenizowanej sekwencji $X = (x_0, x_1, \ldots, x_t)$, perplexity X jest wyrażona wzorem:

$$\text{PPL}(X) = \exp\left\{-\frac{1}{t}\sum_{i=1}^{t}\log p_\theta(x_i \mid x_{<i})\right\}, \tag{3.2}$$

gdzie $\log p_\theta(x_i \mid x_{<i})$ jest logarytmem z wartości prawdopodobieństwa i-tego tokena, uwarunkowanego przez poprzednie tokeny $x_{<i}$. Im mniejsza wartość perplexity dla modelu, tym lepiej ocenia się jego zdolność do predykcji tokenów.

3.5. PUNKTY KONTROLNE

Punkty kontrolne (eng. *checkpoints*) [20] to zapis stanu całego modelu wykonywany podczas procesu uczenia się modelu. Punkty kontrolne zazwyczaj zawierają informacje o wagach, konfiguracji treningu oraz stanie optymalizatora. Zapis punktów kontrolnych umożliwia wznowienie treningu od ostatniego zapisanego stanu modelu, bez utraty dotychczas uzyskanych postępów treningu.

4. NANOGPT

4.1. OPIS NANOGPT

NanoGPT to minimalistyczny model językowy typu GPT stworzony przez *Andreja Karpathy'ego*. Zarówno kod jak i pełen proces tworzenia wraz z tłumaczeniem został opublikowany online. Głównym motywem powstania tego projektu jest utworzenie niewielkiego modelu od podstaw, który umożliwia użytkownikom eksplorowanie technologii przetwarzania języka naturalnego przy ograniczonych zasobach obliczeniowych. Do uruchomienia modelu wystarcza sam procesor, jednakże dla osiągnięcia lepszej wydajności zaleca się korzystanie z karty graficznej obsługującej *CUDA*.

Model używa bibliotekę PyTorch [6], która znajduje zastosowanie w dziedzinie uczenia maszynowego. Dodatkowo projekt wykorzystuje kilka kluczowych bibliotek z wcześniej przedstawionej platformy *Hugging Face*:

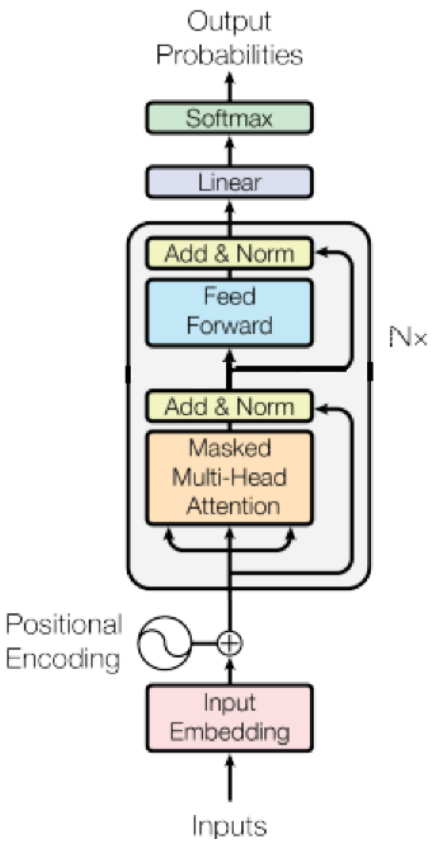
— Transformers, służącej do pobierania wstępnie wytrenowanych punktów kontrolnych modelu językowego, należącego do rodziny modeli *decoder-only*, GPT-2, [31]
— Datasets, w celu uzyskania dostępu do zbiorów danych testowych.

Model używa również biblioteki Tiktoken [7] do tokenizacji oraz NumPy [4] do operacji numerycznych. Dzieło *Andrej'a Karpathy'ego* wspiera również trening i dostrajanie modelu.

4.2. ARCHITEKTURA NANOGPT

NanoGPT wykorzystuje przedstawioną w rozdziale 2 architekturę Transformer. Jednakże, podobnie jak inne modele GPT jest on typu *decoder-only*. Jak nazwa wskazuje, polega wyłącznie na mechanizmie dekodera do generowania tekstu na podstawie wcześniejszych tokenów, bez bez potrzeby przetwarzania dodatkowych informacji z kodera.
Modele typu decoder-only nie wykorzystują kodera, co oznacza, że nie potrzebują dodatkowej warstwy atencji do dekodowania sekwencji przetworzonej przez koder. W rezultacie każda z warstw stosu dekodera dysponuje jedynie jedną maskowaną, wielogłowową podwarstwą samoatencji.

Opisywana architektura została przedstawiona na rys. 4.1



Rysunek 4.1: Architektura decoder-only [15].

NanoGPT pozwala na reprodukcje modelu GPT-2 zawierającego 1.3 miliarda parametrów poprzez trenowanie *NanoGPT* na danych *OpenTextWeb* [5]. Opisywany proces jest jednak niesamowicie czasochłonny i wymaga użycia bardzo dobrego sprzętu. Jednakże równie interesującym zagadnieniem jest dostrajanie wcześniej wytrenowanych punktów kontrolnych pochodzących z modeli GPT-2 Small [2], GPT-2 Medium [28], GPT-2 Large [27] oraz GPT-2 XL [29]. Ten sposób trenowania jest zdecydowanie szybszy i sprawdza się przy mniejszych zasobach obliczeniowych. *NanoGPT* używa optymalizatora *AdamW* [1]. Parametry definiujące liczbę warstw, liczbę głów, rozmiar wektora osadzeń są zależne od rodzaju modelu, który reprodukuje nanoGPT.

Model	**Liczba warstw**	**Liczba głów**	**Rozmiar wektora osadzeń**
GPT-2 Small	12	12	768
GPT-2 Medium	24	16	1024
GPT-2 Large	36	20	1280
GPT-2 XL	48	32	1600

Tabela 4.1: Parametry modeli GPT-2

5. ZABEZPIECZANIE MODELI JĘZYKOWYCH

5.1. KORZYŚCI ZWIĄZANE Z ZABEZPIECZANIEM MODELI JĘZYKOWYCH

W dobie popularności narzędzi generujących tekst, coraz częściej można natknąć się na teksty podpisane czyimś imieniem, będące jednocześnie produktem modeli językowych. Nieraz ciężko określić, czy tekst został napisany przez człowieka, czy też został wytworzony przy użyciu sztucznej inteligencji. Dzięki zabezpieczeniom możliwa jest identyfikacja tekstów wytworzonych przez modele językowe. Używanie zabezpieczeń chroni własność intelektualną, zapobiega plagiatom i różnego rodzaj nadużyciom.

5.2. PROSTA KONCEPCJA ZABEZPIECZENIA

W celu zabezpieczenia modelu, stosuje się cyfrowy znak wody - **Watermark**. Wygenerowany tekst jest znakowany w sposób niewidoczny dla ludzi, a jednocześnie możliwe jest ustalenie czy tekst został wygenerowany przez zadany model językowy. Ten rozdział został poświęcony omówieniu algorytmów stosowanych do znakowania tekstu cyfrowym znakiem wodnym. Przedstawione algorytmy zostały opublikowane w pracy naukowej *A watermark for large language models* [16] w maju 2024 roku.

5.3. PROBLEMY ZWIĄZANE Z ZABEZPIECZANIEM MODELI JĘZYKOWYCH

Teksty o niskiej entropii stanowią problem w kwestii znakowania przy użyciu *watermarka*. Wynika to z faktu, że tokeny występujące w sekwencji silnie determinują wystąpienie kolejnych tokenów. Z tej przyczyny niezwykle ciężko jest jednoznacznie stwierdzić czy tekst został wygenerowany przez człowieka czy też model językowy. Kolejną trudnością jest samo wprowadzenie watermarka w takiej sekwencji. Ze względu na niską entropie, niektóre słowa nie są w stanie być zastąpione bez znacznej ingerencji w znaczenie całości tekstu. Jakakolwiek zmiana wprowadzona w tego typu tekście skutkuje znacznym wzrostem perplexity, co jest równoznaczne z obniżeniem jakości generowanego tekstu.

5.4. SILNY ZNAK WODNY

5.4.1. Koncepcja

Algorytm zakłada podzielenie słownika tokenów na dwie listy - czerwoną i zieloną. Pierwsza z nich przechowuje tokeny, których użycie przez model jest zabronione. Natomiast druga lista przechowuje tokeny, które model może wykorzystywać podczas procesu generowania tekstu.
Tokeny są dzielone w sposób pseudolosowy używając generatora, którego ziarno aktualizuje się każdorazowo po wygenerowaniu nowego tokenu przez model. Jego wartość jest ustalana na podstawie zahashowanej wartości ostatnio wygenerowanego tokenu. Reprezentacja przedstawiona na rys. 5.1 ukazuje przykładowy podział tokenów na dwie listy oraz odpowiadające im prawdopodobieństwa.

Rysunek 5.1: Przykładowa reprezentacja czerwonej i zielonej listy.

5.4.2. Algorytm silnej czerwonej listy

Algorytm wykorzystuje słownik tokenów $\mathcal{V}$, prompt długości $\mathcal{N}$ zawierający tokeny oznaczone ujemnymi indeksami $s^{(-N_p)}, \ldots, s^{(t-1)}$ oraz sekwencję $\mathcal{T}$ tokenów wygenerowanych przez model oznaczonych indeksami dodatnimi $s^{(0)}, s^{(1)}..s^{(\mathcal{T})}$. Wartość indeksu reprezentuję pozycję tokenu w przetwarzanej sekwencji, zatem token z niższym indeksem znajduje się wcześniej w tekście. Przedstawiony algorytm 1 bazuje na wcześniej przedstawionej koncepcji.

Algorithm 1 Algorytm ciężkiej czerwonej listy [16]

Require: prompt: tokeny $s^{(-N_p)}, \ldots, s^{(-1)}$
parametr rozmiaru zielonej listy $\gamma \in (0, 1)$

1: **for** $t = 0, 1, ...\mathcal{T}$ **do**

2: Użyj model językowy do przetworzenia sekwencji tokenów $s^{(-N_p)}, \ldots, s^{(t-1)}$ w celu uzyskania wektora prawdopodobieństwa $p(t)$ dla całego słownika $\mathcal{V}$.

3: Oblicz wartość funkcji hashującej dla tokenu $s^{(t-1)}$ i użyj jej jako ziarna dla generatora pseudolosowego.

4: Używając tego generatora, podziel losowo słownik na *zieloną listę* G o rozmiarze $\gamma|\mathcal{V}|$ oraz *czerwoną listę* R o rozmiarze $(1-\gamma)|\mathcal{V}|$.

5: Wybierz $s^{(t)}$ z G, zapobiegając wygenerowaniu tokenu z *czerwonej listy* R.

6: **end for**

5.5. LEKKI ZNAK WODNY

5.5.1. Koncepcja

We wcześniej omawianym algorytmie występuje problem z zabezpieczaniem tekstów o niskiej entropii. Jednak istnieje możliwość poprawienia wyżej przedstawionej koncepcji, minimalizując ryzyko znacznego wzrostu perplexity. Otóż, w dalszym ciągu należy dzielić słownik tokenów na dwie listy.

Jednak tym razem, celem jest zwiększenie prawdopodobieństw dla tokenów znajdujących się na zielonej liście. Efekt ten można uzyskać, poprzez dodanie stałej δ do ich logitów. Wspomniana wcześniej stała δ określana jest jako parametr mocy. Warto podkreślić, że w tym przypadku przynależność tokena do czerwonej listy nie wyklucza jego wyboru w procesie generacji.

Modele językowe stosujące wymieniony algorytm, przetwarzając sekwencje o wysokiej entropii prawie zawsze nie wybierają tokenów należących do czerwonej listy, natomiast zasada ta jest ignorowana w przypadku, kiedy wybór danego elementu jest konieczny. Zastosowanie tej techniki znacząco wpływa na poprawę jakości tekstów generowanych przez model stosujący cyfrowe znaki wodne, co jest jednoznaczne z obniżeniem perpexity.

5.5.2. Algorytm lekkiej czerwonej listy

Poniżej prezentuje się algorytm 2 bazujący na wcześniej przedstawionej koncepcji. Wszelkie przyjęte konwencje są zgodne z tymi stosowanymi w *algorytmie 1*. Rozmiar *zielonej listy* ustalany jest za pomocą parametru γ, natomiast wartość δ determinuje parametr mocy.

Algorithm 2 Generowanie tekstu z lekką czerwoną listą [16]

Require: prompt : tokeny $s^{(-N_p)}, \ldots, s^{(-1)}$
parametr rozmiaru zielonej listy: $\gamma \in (0, 1)$
parametr mocy: $\delta > 0$

1: **for** $t = 0, 1, \ldots \mathcal{T}$ **do**

2: Użyj model językowy do przetworzenia sekwencji tokenów $s^{(-N_p)}, \ldots, s^{(t-1)}$, w celu uzyskania wektora logitów $l^{(t)}$ dla całego słownika $\mathcal{V}$.

3: Oblicz wartość funkcji hashującej dla tokenu $s^{(t-1)}$ i użyj jej jako ziarna dla generatora pseudolosowego.

4: Używając tego generatora, losowo podziel słownik na *zieloną listę* G o rozmiarze $\gamma|\mathcal{V}|$ oraz *czerwoną listę* R o rozmiarze $(1-\gamma)|\mathcal{V}|$.

5: Dodaj δ do każdego logitu, którego odpowiadający token jest na zielonej liście. Zastosuj funkcję softmax na zmodyfikowanych logitach, aby uzyskać rozkład prawdopodobieństwa $\hat{p}^{(t)}$ nad słownikiem $\mathcal{V}$:

$$\hat{p}_k^{(t)} = \begin{cases} \frac{\exp(l_k^{(t)}+\delta)}{\sum_{i\in R}\exp(l_i^{(t)})+\sum_{i\in G}\exp(l_i^{(t)}+\delta)}, & k \in G \\ \frac{\exp(l_k^{(t)})}{\sum_{i\in R}\exp(l_i^{(t)})+\sum_{i\in G}\exp(l_i^{(t)}+\delta)}, & k \in R \end{cases}$$

6: Wybierz kolejny token $s^{(t)}$ za pomocą rozkładu $\hat{p}^{(t)}$.

7: **end for**

5.6. DETEKCJA CYFROWEGO ZNAKU WODNEGO

Ten podrozdział zawiera opis procesu wykrywania cyfrowego znaku wodnego znając słownik wszystkich tokenów $\mathcal{V}$, funkcję hashującą oraz parametr γ. W każdym kroku iteracji, możliwe jest dokładne odtworzenie czerwonej i zielonej listy, a co za tym idzie - możliwe jest stwierdzenie do której z nich należał wygenerowany token. Zatem, da się ustalić ile spośród wygenerowanych tokenów było członkami zielonej listy.
Przypuszczając, że tekst został napisany przez człowieka, należy spodziewać się, że będzie się on składał z około $1-\gamma$ tokenów z listy czerwonej i γ tokenów z listy zielonej. W przypadku kiedy tekst został wygenerowany przez zabezpieczony model, ta statystyka nie zostanie spełniona.

Liczba tokenów w sekwencji wytworzonej przez człowieka należących do zielonej listy może być modelowana jako zmienna losowa. Zatem wartość oczekiwana liczby tokenów z listy zielonej dla tekstów pochodzenia naturalnego wynosi $\gamma \cdot \mathcal{T}$, a wariancja $\mathcal{T} \cdot \gamma \cdot (1-\gamma)$ W celu formalnego stwierdzenia występowania *watermarka* wykorzystuje się *test z* [32] do oceny hipotezy zerowej, która zakłada, że tekst został wygenerowany przez człowieka. *Z-score*, czyli miara określająca, jak bardzo konkretna wartość różni się od

średniej, wyrażona w jednostkach odchylenia standardowego, dla tego testu jest obliczany według poniżej przedstawionego wzoru

$$z = \frac{(|s|_G - \gamma \cdot \mathcal{T})}{\sqrt{\mathcal{T} \cdot \gamma \cdot (1-\gamma)}}. \tag{5.1}$$

Hipoteza zerowa jest odrzucana, jeżeli wartość z-score jest większa niż ustalony próg.

5.7. PROBLEMY ZWIĄZANE Z DETEKCJĄ LEKKIEGO ZNAKU WODNEGO

5.7.1. Spike entropy

Spike entropy [16] to miara której celem jest ustalenie jak bardzo rozkład prawdopodobieństwa skupiony jest na niewielkiej licznie tokenów. Spike entropy dla sekwencji $\mathcal{T}$ tokenów wyraża się wzorem:

$$S(p, z) = \sum_{k=1}^{\mathcal{T}} \frac{p^k}{1 + zp^k}. \tag{5.2}$$

Minimalna wartość *Spike entropy* równa $\frac{1}{1-z}$ uzyskiwana jest kiedy rozkład prawdopodobieństwa skupia się jedynie na jednym tokenie, któremu przypisuje się prawdopodobieństwo równe 1, natomiast wszystkie inne tokeny uzyskują prawdopodobieństwo równe 0. Maksymalna wartość *spike entropy* wynoszącą $\frac{\mathcal{T}}{\mathcal{T}+z}$ uzyskiwana jest dla równomiernie rozłożonego rozkładu w którym każdy z tokenów uzyskuje prawdopodobieństwo równe $\frac{1}{\mathcal{T}}$.

Dla dużych wartości z:

— Jeśli $p_k > \frac{1}{z}$, to

$$\frac{p_k}{1 + zp_k} \approx \frac{1}{z}.$$

— Jeśli $p_k < \frac{1}{z}$, to

$$\frac{p_k}{1 + zp_k} \approx 0.$$

Zatem *spike entropy* można definiować jako miarę liczby tokenów przyjmujących prawdopodobieństwo powyżej $\frac{1}{z}$.

Stosując algorytm *silnej czerwonej listy* do logitów tokenów znajdujących się na liście zielnej dodawana jest wartość δ. Dla $\alpha = \exp(\delta)$ można założyć, że podczas obliczania prawdopodobieństwa wartości prawdopodobieństw dla tokenów z zielonej listy prezentują się następująco:

$$p_k = \frac{\alpha \cdot \exp(l_k^{(t)})}{\sum_{i \in R} \exp(l_i^{(t)}) + \sum_{i \in G} \alpha \cdot \exp(l_i^{(t)})}. \tag{5.3}$$

5.7.2. Wartość oczekiwana rozmiaru zielonej listy

W pracy naukowej *A watermark for large language models* [16] wyprowadzony został wzór na ograniczenie dolne wartości oczekiwanej rozmiaru zielonej list. Prezentuje się on następująco :

$$\mathbb{E}\,|s|_G \geq \frac{\gamma\alpha T}{1+(\alpha-1)\gamma} S^\star, \tag{5.4}$$

gdzie $S^\star$ ustalane jest według poniżej przedstawionej nierówności:

$$\frac{1}{T}\sum_t S\left(p(t), \frac{(1-\gamma)(\alpha-1)}{1+(\alpha-1)\gamma}\right) \geq S^\star. \tag{5.5}$$

5.7.3. Wnioski

Wyżej zostało pokazane, że *spike entropy* ma wpływ na wartość oczekiwaną wystąpień liczby zielonych tokenów w tekście wygenerowanym z zastosowaniem algorytmu *lekkiej czerwonej listy*.

Wartość oczekiwana jest również zależna od dobranych parametrów γ i δ. Wszystkie te czynniki sprawiają, że możliwe jest błędne zaklasyfikowanie tekstu napisanego przez człowieka jako tekst wygenerowany przez model językowy.

5.8. WPŁYW CYFROWEGO ZNAKU WODNEGO NA JAKOŚĆ GENEROWANEGO TEKSTU

Najbardziej istotną intencją omawianego mechanizmu jest znakowanie generowanego tekstu bez znacznej utraty jakości. Rozpatrując oba omówione algorytmy, należy zauważyć, że to właśnie drugi z nich jest w stanie wpłynąć na lepsze wyniki generacji.

6. ROZWÓJ I DOSTRAJANIE MODELU NANOGPT

6.1. WPROWADZENIE

Zanim przedstawione zostaną algorytmy zabezpieczające modele językowe, szczególną uwagę należy zwrócić na sposób generowania tekstu przez NanoGPT. Funkcja generująca tekst, przyjmuje kilka argumentów - sekwencje tokenów wejściowych, temperaturę, parametr γ i δ ale także parametr *max_new_tokens* ustalający maksymalną długość generowanej sekwencji oraz parametr *top_k*, ograniczający liczbę logitów uwzględnianych podczas losowego wyboru kolejnego tokenu.

Najpierw przez model przepuszczana jest sekwencja tokenów, w konsekwencji czego otrzymywane są logity. Logity mogą zostać następnie przeskalowane przez parametr temperatury, a także ograniczyć do *top_k* najwyższych wartości. Kolejno stosowana jest funkcja softmax, która przekształca logity w prawdopodobieństwa. Natomiast kolejny token wybierany jest losowo z rozkładu wielomianowego i dołączany jest do istniejącej sekwencji. Proces ten jest powtarzany iteracyjnie *max_new_tokens* razy.

Naturalnie można zauważyć, że to właśnie funkcja generująca tekst jest miejscem gdzie należy wprowadzić modyfikacje w celu zabezpieczenia modelu cyfrowymi znakami wodnymi.

6.2. SILNY ZNAK WODNY

Ta sekcja skupia się na omówieniu implementacji *algorytmu silnej czerwonej listy* w modelu NanoGPT. Metoda zakłada wprowadzenie modyfikacji na poziomie wektorów prawdopodobieństw do których model ma dostęp w trakcie generowania tokenów.
Jednakże w celu dogłębnej analizy algorytmu, należy przedstawić funkcję *get_random_indices*, skupiającej się na generowaniu listy pseudolosowych indeksów o określonej przez parametry *vector_size* oraz γ długości *zielonej listy*.

Pseudolosowość wynika z wykorzystania generatora pseudolosowego, którego ziarno jest określone przez wartość funkcji hashującej obliczanej na podstawie ostatniego wygenerowanego tokena w sekwencji.

Stosowana w tym celu metoda *get_hash_value* przyjmuje jako argument osadzenie tokenu, następnie identyfikuje odpowiadający mu token na podstawie podanego wektora oraz oblicza wartość funkcji hashującej, korzystając z algorytmu SHA-256.

Implementacja algorytmu nie zakłada przechowywania dwóch osobnych list z tokenami lub ich osadzeniami. Przechowywanie list tokenów doprowadziłoby do zwiększenia zużycia pamięci. Listy nie są potrzebne, ponieważ możliwe jest wykorzystanie indeksów tokenów w wyznaczanym wektorze prawdopodobieństw.

Główna idea zakłada podział tokenów wykorzystując w tym celu wektor prawdopodobieństw. Znając wartość funkcji hashującej dla ostatnio wygenerowanego tokenu, możliwe jest podzielenie indeksów wektora prawdopodobieństw na dwa zbiory:

— Pierwszy ze zbiorów jest interpretowany jako lista zielona. Każdy indeks wektora prawdopodobieństwa odpowiada dokładnie jednemu tokenowi. Zatem token odpowiadający temu indeksowi będzie traktowany jako element listy zielonej.
— Drugi ze zbiorów jest interpretowany jako lista czerwona. Tokeny odpowiadające tym indeksom będą traktowane jako elementy listy czerwonej.

Możliwe jest manipulowanie rozmiarami tych list za pomocą parametry γ. Algorytm wybierający indeksy odpowiadające tokenom z listy zielonej spośród indeksów zadanego wektora przedstawiony jest poniżej

Algorithm 3 get_random_indices

Require: Rozmiar wektora: $vector_size$
identyfikator tokenu: $token_id$
współczynnik rozmiaru *zielonej listy*:$\gamma \in (0, 1)$

1: $green_list_size \leftarrow$ math.floor$(probs_vector_size \cdot \gamma)$

2: $random.seed($get_hash_value$(token_id))$ //Ustawienie ziarna losowości na podstawie wartości funkcje hashującej dla tokenu o identyfikatorze *token_id*

3: $random_indices \leftarrow$ random.sample(range(vector_size), green_list_size) //Wybór losowych indeksów, które odpowiadają tokenom z listy zielonej

4: **return** $random_indices$

Poniżej widnieje algorytm generujący zieloną i czerwoną listę. W jego konstrukcji używane są wyżej omówione funkcje pomocnicze.

Algorithm 4 Generowanie zielonej i czerwonej listy

Require: Wektor prawdopodobieństw: $probs$
identyfikator tokenu: $token_id$
parametr rozmiaru zielonej listy: $\gamma \in (0, 1)$

1: $probs_vector_size \leftarrow$ długość wektora prawdopodobieństw $probs$

2: $all_indices \leftarrow$ tablica zawierająca wszystkie indeksy wektora $probs$

3: $random_green_list_indices \leftarrow$ losowe indeksy wybierane za pomocą funkcji $get_random_indices(probs_vector_size, token_id, \gamma)$

4: $red_list_indices \leftarrow$ list($all_indices$ - set($random_green_list_indices$))

5: $probs[red_list_indices] \leftarrow 0$

6: **return** Zaktualizowany wektor prawdopodobieństw $probs$

6.3. LEKKI ZNAK WODNY

Algorytm *lekkiej czerwonej listy* bazuje na tej samej koncepcji, przy czym również nie wymaga przechowywania list tokenów. Jednakże w tym przypadku modyfikacja wartości odbywa się na poziomie logitów. Zatem opisywany algorytm prezentuje się następująco:

Algorithm 5 Generowanie lekkiej zielonej i czerwonej listy

Require: Wektor logitów: logits
Idenfyfikator tokenu: token_id
Parametr rozmiaru *zielonej listy*:$\gamma \in (0, 1)$
Parametr mocy: δ

1: $vector_size \leftarrow$ długość wektora logitów

2: $random_green_list_indices \leftarrow$ get_random_indices($vector_size$, $token_id, \gamma$)

3: $logits[random_green_list_indices] \leftarrow logits[random_indices] + \delta$

4: **return** logits

6.4. DETEKCJA CYFROWEGO ZNAKU WODNEGO

Detekcja cyfrowego znaku wodnego może odbywać się bez bezpośredniego dostępu do modelu. Konieczny jest natomiast dostęp do tokenizatora używanego przez model oraz znajomość stosowanej funkcji hashującej. Tokenizator jest niezbędny do przekształcenia analizowanej sekwencji w tokeny, których indeksy są kluczowe w procesie detekcji *watermarka*. Ponadto to narzędzie dostarcza informacje na temat długości słownika tokenów $\mathcal{V}$. Detekcja odbywa się iteracyjnie, każdy z tokenów analizowany jest osobno według kolejności występowania w sekwencji. Wszystkie tokeny (oprócz początkowego) posiadają swojego poprzednika, który jest wykorzystywany do odtworzenia listy indeksów listy

zielonej *green_list_indices*. Kolejnym etapem jest przeprowadzenie weryfikacji, do której listy należy analizowany token. W przypadku gdy token należy do listy list, dochodzi do zwiększenia licznika *green_list_counter*, jak sama nazwa wskazuje, reprezentuje rozmiar *zielonej listy*. Po poddaniu analizie wszystkich tokenów w sekwencji, wyliczana zostaje *z-score*.

Algorithm 6 Detekcja watermarka

Require: Sekwencja tokenów: idx
Rozmiar słownika: vocab_size
Parametr wyznaczający długość zielonej listy: γ

```
green_list_counter ← 0
seq_len ← długość przetwarzanej sekwencji idx
for i = 0 to seq_len − 1 do
    idx_next ← idx[i + 1]
    green_list_indices ← get_random_indices(vocab_size, idx[i].item(), γ)
    if idx_next.item() ∈ green_list_indices then
        green_list_counter ← green_list_counter + 1
    end if
end for
z_score ← (green_list_counter − γ·seq_len) / √(seq_len·γ·(1−φ))
Return z_score
```

6.5. ZŁOŻONOŚĆ OBLICZENIOWA

Po omówieniu implementacji przedstawionych algorytmów, należy również skupić się na ich złożoności obliczeniowej. Zatem tabla przedstawiająca złożoności obliczeniowe dla zadanych algorytmów, długości przetwarzanej sekwencji $\mathcal{N}$ i rozmiaru słownika $\mathcal{V}$ prezentuje się następująco:

	Lekka czerwona lista	**Silna czerwona lista**
Algorytm	$O(\mathcal{V})$	$O(\mathcal{V})$
Detekcja	$O(\mathcal{N} \cdot \mathcal{V})$	$O(\mathcal{N} \cdot \mathcal{V})$

Tabela 6.1: Złożoności obliczeniowe

6.6. DOSTRAJANIE MODELU

6.6.1. Transfer wag wytrenowanego modelu

Trenowanie modeli językowych jest niezwykle czasochłonnym i złożonym obliczeniowo procesem. Możliwe jest natomiast pozyskanie wag wytrenowanego modelu i zaaplikowanie ich w innym modelu. Zabieg ten jest znany jako *transfer wag*. W kontekście dostrajania modelu NanoGPT transfer wag dokonywany jest z wytrenowanego wcześniej modelu GPT-2. Jest to forma pewnego przekazania wiedzy, która pozwala na efektywne wykorzystanie wcześniej wyuczonych wag.

Korzystając z biblioteki *Hugging Face Transformers* tworzona jest instancja modelu GPT-2 wraz z załadowanymi wytrenowanymi wagami. Chcąc uzyskać dostęp bezpośrednio do wag tego modelu, wykorzystuje się funkcje *state_dict()* z biblioteki *PyTorch*, która zwraca stan modelu w postaci słownika. Finalnie wartości te są przenoszone do modelu NanoGPT.

6.6.2. Dostrajanie NanoGPT

W celu przeprowadzenia *fine-tuningu* modelu NanoGPT zawierającego wytrenowane wcześniej wagi modelu GPT-2 Large, należy go jedynie dotrenować wykorzystując wybrany zbiór danych tekstowych. Model NanoGPT został dostrojony używając zbioru danych *wikitext* w wersji *wikitext-2-raw-v1* [24] importowanego z biblioteki *Datasets* z platformy *Hugging Face* [3]. Źródłem tych danych są zweryfikowane artykuły z *Wikipedii*.

7. BADANIA I ANALIZA WYNIKÓW

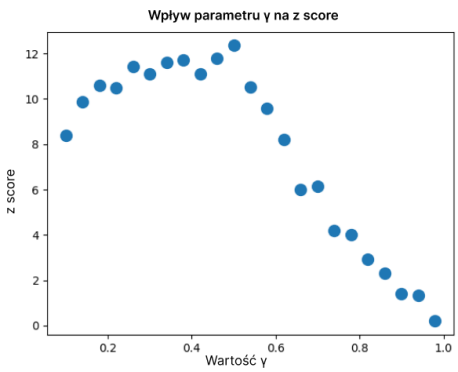
7.1. PRZEPROWADZONE BADANIA

W celu dobrania optymalnych parametrów używanych do zabezpieczania modeli cyfrowym znakiem wodnym, należy przeprowadzić serię badań mających na celu ustalić wpływ używanych parametrów na uzyskiwane wyniki generacji i moc cyfrowego znaku wodnego. W ten sekcji przedstawiono zarówno przeprowadzone badania, jak i ich rzetelną analizę. Przedstawiona w rozdziale 5 analiza algorytmów *watermarkingu* opowiada się za wyższością algorytmu *lekkiej czerwonej listy*, zatem domyślnie opisane eksperymenty zostały przeprowadzone przy jego użyciu. Wszystkie przedstawione badania zostały przeprowadzone na sekwencjach o długości 200 tokenów, używając przy tym próbkowania wielomianowego i temperatury 0.8. Natomiast do wyznaczenia wartości *perplexity* użyto modelu językowego GPT2.

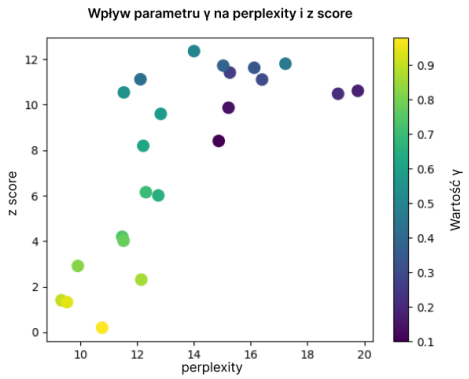
7.1.1. Badanie wpływu wielkości zielonej listy na moc znaku wodnego i jakość tekstu

7.1.1.1. Rezultat

Przedstawiony w rozdziale 5 parametr γ przyjmuje wartości należące do przedziału $(0, 1)$ i jest odpowiedzialny za ustalenie rozmiaru zielonej listy. Ten podrozdział prezentuje rezultaty badań wpływu γ na uzyskiwane wyniki przy stałym parametrze $\delta = 5$.



Rysunek 7.1: Wpływ parametru γ na *z-score*



Rysunek 7.2: Wpływ parametru γ na *z-score* i *perplexity*

7.1.1.2. Analiza

Przedstawione wyniki zwracają uwagę na kilka ważnych aspektów. Przede wszystkim pod kątem mocy zabezpieczenia cyfrowym znakiem wodnym najlepsze rezultaty uzyskiwane są w okolicach $\gamma \approx 0.5$.
Kolejnym godnym uwagi zagadnieniem jest uzyskiwane w badaniach *perplexity*. Otóż wraz ze wzrostem rozmiaru *zielonej listy*, wynikających z wzrostu wartości γ, generowane teksty charakteryzują się niższymi wartościami *perplexity*, co za tym idzie - jakość pozyskiwanych tekstów jest lepsza. Zmniejszenie rozmiaru *zielonej listy* prowadzi do pogorszenia jakości tekstów generowanych przez model, ponieważ ograniczona liczba tokenów należących do *zielonej listy*, preferowanych przez model podczas procesu generowania przy użyciu algorytmu *lekkiej czerwonej listy*, wpływa negatywnie jego wydajność.

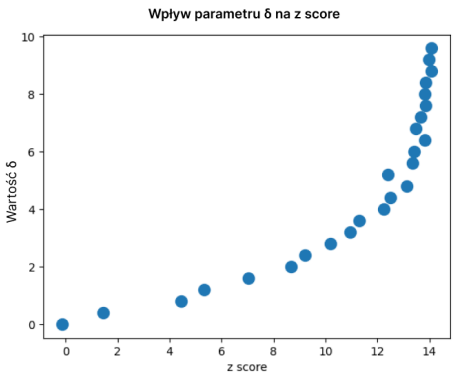
7.1.2. Aspekt matematyczny

Rys. 7.1 przedstawia nietypowy kształt wykresu wartości uzyskiwanych przez *z-score*. Dla niskich wartości parametru γ zauważalne są duże wartości *z-score*. Ponadto wartości te początkowo rosną do uzyskania przez parametr γ wartości ≈ 0.5. Obserwowany kształt wykresu na tym przedziale wynika z rozbieżności pomiędzy *wartością oczekiwaną*, a rzeczywistą liczbą wystąpień tokenów z listy zielonej.
Wzmocnienie logitów tokenów z listy zielonej przez parametr mocy δ prowadzi do zwiększenia preferencji modelu do ich wyboru, co skutkuje dużą różnicą między *wartością oczekiwaną* a rzeczywistą liczbą wystąpień tokenów z listy zielonej. W rezultacie otrzymywane są duże wartości *z-score*.
Natomiast przyczyna wzrostu wartości *z-score* na tym przedziale jest związana bezpośrednio z entropią tekstów, a także opisaną w rozdziale 5, *spike entropy*. Otóż, algorytm 2 nie blokuje wyboru tokenów z listy czerwonej, jednakże manipuluje rozkładem prawdopodobieństw w ten sposób, żeby zwiększyć preferencje modelu względem tokenów z listy zielonej. Jeżeli entropia jest niska i po zastosowaniu wzmocnienia δ najwyższą wartość logitu ma token znajdujący się na czerwonej liście, to nadal ma on największe prawdopodobieństwo bycia wybranym przez model. Jednocześnie należy wziąć pod uwagę, że im większy rozmiar listy zielonej, tym mniej prawdopodobne jest wystąpienie sytuacji w której token z czerwonej listy zostaje wybrany przez model w procesie generacji.
Z kolei dla wyższych wartości γ zauważalny jest spadek wartości *z-score*. Sytuacja ta występuje ze względu na to że, generowana liczba zielonych tokenów jest coraz bardziej zbliżona do *wartości oczekiwanej* ze względu na zwiększający się rozmiar listy zielonej.

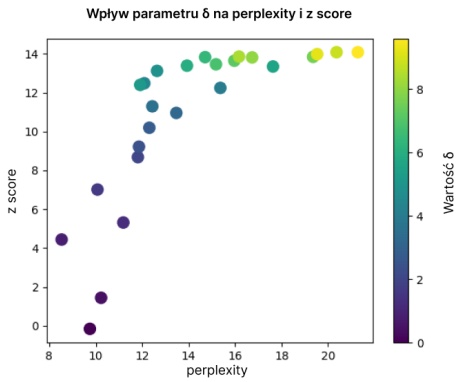
7.1.3. Badanie wpływu parametru mocy na moc znaku wodnego i jakość tekstu

Parametr δ odpowiedzialny za przesunięcie w logitach dla tokenów znajdujących się na *liście zielonej*, przyjmuje wartości zawierające się w przedziale$(0, \infty)$. Im wyższa

wartość δ tym bardziej model preferuje tokeny wchodzące w skład *listy zielonej*. Poniżej przedstawione są rezultaty badań wpływu parametru δ na takie miary jak *perplexity* i *z-score*.



Rysunek 7.3: Wpływ parametru δ na *z-score*



Rysunek 7.4: Wpływ parametru δ na *perplexity* i *z-score*

7.1.3.1. Analiza

Pierwszy z przedstawionych wykresów jasno pokazuje, że wzrost parametru δ skutkuje wzrostem *z-score*. Z tego wynika, że wraz ze wzrostem prawdopodobieństw przypisanym tokenom występujących na *liście zielonej*, co prowadzi do częstszego ich wybierania. W wyniku tego działania rośnie również *z-score*. W efekcie zwiększa się moc cyfrowego znaku wodnego, co prowadzi do łatwiejszego wykrywania oznakowanych tekstów.

Wskutek zwiększania się wartości parametru δ rośnie wartość *perplexity*. Zachowanie to wynika z faktu, że im wyższe wartości δ tym tokeny z zielonej listy są bardziej preferowanym wyborem podczas procesu generacji. Prowadzi to do rzadszego wyboru tokenów znajdujących się na *liście czerwonej*, nawet jeśli ma to negatywny wpływ na wynik. Zatem, wzrost wartości *perplexity* jest efektem nadmiernej preferencji tokenów wchodzących w skład *listy zielonej*, przy jednoczesnym niedostatecznym uwzględnianiu tokenów znajdujących się na *liście czerwonej*.

Zatem zwiększanie tego parametry wpływa korzystnie w kontekście mocy cyfrowego znaku wodnego, jednocześnie wpływając negatywnie na jakość generacji.

7.1.4. Aspekt matematyczny

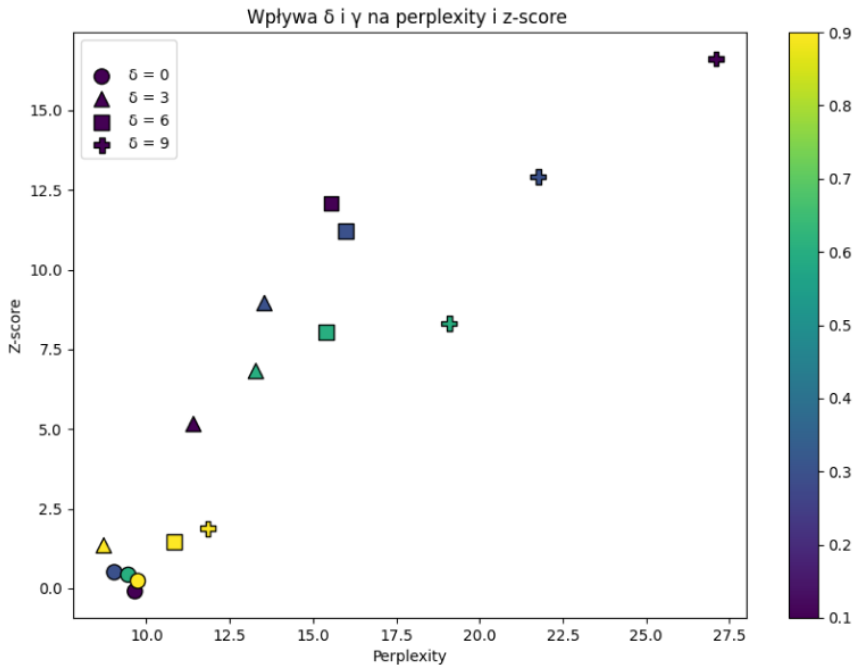
Nadmierna preferencja tokenów z listy zielonej prowadzi do wzrostu wartości *perplexity* dla wzrastających wartości parametru δ, co jest pokazane na wykresie 7.4.

Oczywisty jest także wzrost wartości *z-score* wynikający ze zwiększonej preferencji modelu do wybierania tokenów z listy zielonej. W rezultacie prowadzi to do zwiększania

różnicy między uzyskiwaną liczbą wystąpień tokenów z listy zielonej a ich wartością oczekiwaną.

7.1.5. Jednoczesne badanie parametru γ i δ

W ramach ujednolicenia uzyskanych wyników, wygenerowany został również wykres zależności obu parametrów - γ i δ na otrzymywaną wartość perplexity i z-score.

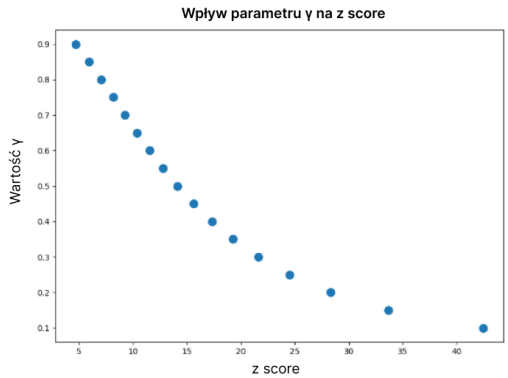


Rysunek 7.5: Wpływ γ i δ na otrzymywane wartości *z-score* i *perplexity*

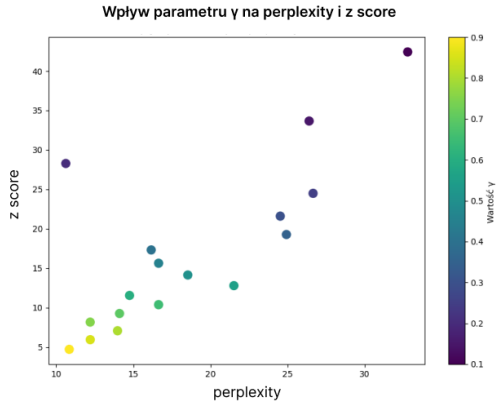
Wykres przedstawiony na rys. 7.5 dodatkowo potwierdza wcześniej uzyskane wyniki. Jak można zauważyć, *z-score* i *perplexity* są ściśle powiązane z oboma parametrami.

7.1.6. Badania algorytmu silnej czerwonej listy

Rozdział 5 obszernie opisuje przyczynę górowania *algorytmu lekkiej czerwonej listy* nad *algorytmem ciężkiej czerwonej listy*. W celu obserwacji na jak dużą skale występuje zjawisko pogorszenia się wartości perplexity wskutek zastosowania algorytmy *silnej czerwonej listy*, wygenerowany został wykres zależności parametru γ na wartość perplexity i z-score.



Rysunek 7.6: Wpływ parametru γ na *z-score*



Rysunek 7.7: Wpływ parametru γ na *perplexity* i *z-score*

Wykres 7.6 przedstawia zauważalny spadek wartości *z-score* wraz ze wzrostem parametru γ

Natomiast wykres 7.7 wskazuje że, zastosowanie tego algorytmu *silnej czerwonej listy* zwiększa wartość *perplexity* w porównaniu do algorytmu *lekkiej czerwonej listy*. Porównując ten wykres do wykresu 7.4 zauważalny jest wzrost *perplexity* dla odpowiadających sobie wartości parametru γ.
Jednocześnie na rys. 7.7 zauważalna jest poprawa w kontekście uzyskiwanych wartości *perplexity* wraz ze wzrostem γ.

7.1.7. Aspekt matematyczny

Analizując funkcję *z-score* dla sekwencji $\mathcal{T}$ tokenów i parametru $\gamma \in (0, 1)$, gdzie zmienna losowa $\mathcal{X} = \mathcal{T}$ wyraża się wzorem

$$z\text{-}score(\gamma) = \frac{\mathcal{T} - \gamma\mathcal{T}}{\sqrt{\mathcal{T}(1-\gamma)\gamma}}. \tag{7.1}$$

Nietrudno dostrzec, że jest to funkcja monotoniczna stale malejąca na przedziale $(0, 1)$. Dodatkowo można określić granice omawianej funkcji.

$$\lim_{\gamma \to 1^-} \frac{\mathcal{T} - \gamma\mathcal{T}}{\sqrt{\mathcal{T}(1-\gamma)\gamma}} = 0, \tag{7.2}$$

$$\lim_{\gamma \to 0^+} \frac{\mathcal{T} - \gamma\mathcal{T}}{\sqrt{\mathcal{T}(1-\gamma)\gamma}} = \infty. \tag{7.3}$$

Z tej przyczyny, można wnioskować że wartości funkcji zwiększają się dla malejących wartości γ. Przedstawiony powyżej wykres 7.6 wykres we właściwy sposób obrazuje przedstawione zależności.

7.2. ANALIZA WYNIKÓW

Na mocy uzyskanych wyników, eksperymentalnie zostało pokazane, że wzrost wartości parametru δ wpływa korzystnie na uzyskiwany *z-score*, co jest równoznaczne ze wzmocnieniem cyfrowego znaku wodnego. Dla parametru γ najlepszy wynik w kontekście uzyskiwanej mocy cyfrowego znaku wodnego występował dla $\gamma \approx 0.5$.

Wzrost δ wpływał negatywnie na wartość *perplexity*. Natomiast w przypadku parametru γ zwiększenie tej wartości przynosiło korzyści pod kątem uzyskiwanego *perplexity*.

Z tej przyczyny niezmiernie ważne jest dobranie odpowiednich parametrów, tak aby cyfrowy znak wodny był odpowiednio silny, zarazem wyniki generacji były dobrej jakości. Dodatkowo, obserwacje wynikające z przedstawionych wykresów wskazują na spójność uzyskanych rezultatów z matematycznymi przewidywaniami.

PODSUMOWANIE

OMÓWIENIE ZAWARTOŚCI PRACY

W niniejszej pracy przedstawiono szczegółowy opis architektury Transformer, która znajduje zastosowanie wśród współczesnych modeli językowych, takich jak GPT. Omówione zostały zagadnienia związane z tokenizacją, wektorami osadzeń, samoatencją oraz procesem trenowania i dostrajania modeli, a także generowania tekstu, co jest kluczowe w kontekście zrozumienia działania modeli językowych. Szczególną uwagę poświęcono również modelowi NanoGPT.

Jednakże w niniejszej pracy największy nacisk został położony na analizę algorytmów *watermarkingu* stosowanych w celu zabezpieczenia modeli językowych cyfrowym znakiem wodnym. Przedstawiona została implementacja owych algorytmów oraz badania wpływu parametrów na uzyskiwane wyniki. Część eksperymentalna została przeprowadzona na modelu NanoGPT.

EFEKTYWNOŚĆ PRZESTAWIONYCH ALGORYTMÓW

Przedstawione badania algorytmów wykazały, że są one wydajne w kontekście zabezpieczania modeli językowych. Wysoka skuteczność detekcji cyfrowych znaków wodnych, przy nieznacznym negatywnym wpływie na jakość tekstów generowanych przez model, świadczy o dużej użyteczności opisanych w pracy mechanizmów w celu zapewniania bezpieczeństwa i ochrony przed nadużyciami wynikającymi z przywłaszczeń tekstów generowanych przez modele językowe.

WYZWANIA ZWIĄZANE Z IMPLEMENTACJĄ

Implementując przestawione algorytmy *watermarkingu*, należy mieć świadomość że wiąże się to z szeregiem trudności. Najbardziej istotnym problemem jest zachowanie balansu między mocą cyfrowego znaku wodnego, a jakością generowanych wyników. Należy pamiętać, że wraz ze wzrostem mocy, dochodzi do pogorszenia się jakości generowanych tekstów. Dodatkowo, skuteczna implementacja wymaga optymalizacji algorytmów pod kątem wydajności.

BIBLIOGRAFIA

[1] *Adamw*, https://pytorch.org/docs/stable/generated/torch.optim.AdamW.html.

[2] *Gpt-2*, https://huggingface.co/openai-community/gpt2.

[3] *Hugging face documentation*, https://huggingface.co/docs.

[4] *Numpy*, https://numpy.org/.

[5] *Openwebtext*, https://huggingface.co/datasets/Skylion007/openwebtext.

[6] *Pytorch documentation*, https://pytorch.org/docs/stable/index.html.

[7] *Tiktoken*, https://github.com/openai/tiktoken.

[8] 3Blue1Brown, *Attention in transformers, visually explained*, https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi. 2024.

[9] Amanatullah, *Transformer architecture explained*. 2023.

[10] Ashish, V., Noam, S., Niki, P., Jakob, U., Llion, J., Aidan, G., Łukasz, K., *Attention is all you need (transformer) - model explanation (including math), inference and training*, https://arxiv.org/pdf/1706.03762.

[11] Bergmann, D., *What is fine-tuning?*, https://www.ibm.com/topics/fine-tuning.

[12] Colyer, A., *The amazing power of word vectors*, https://blog.acolyer.org/2016/04/21/the-amazing-power-of-word-vectors.

[13] Hugging Face, *Perplexity of fixed-length models*, https://huggingface.co/docs/transformers/perplexity.

[14] Jacob, D., Ming-Wei, C., Kenton, L., Kristina, T., *Bert: Pre-training of deep bidirectional transformers for language understanding*. 2019.

[15] Jamil, U., *Attention is all you need*, https://www.youtube.com/watch?v=bCz4OMemCcA. Ost. dost. 28 maja 2023.

[16] John, K., Jonas, G., Yuxin, W., Jonathan, K., Ian, M., Tom, G., *A watermark for large language models*, https://arxiv.org/pdf/2403.13027.

[17] Karpathy, A., *Nanogpt*, https://github.com/karpathy/nanoGPT.

[18] Karpathy, A., *Neural networks: Zero to hero*, https://karpathy.ai/zero-to-hero.html.

[19] Karpathy, A., *Let's build gpt: from scratch, in code, spelled out*, https://www.youtube.com/watch?v=kCc8FmEb1nY&t=3939s. 2023.

[20] Ku, Wee, K., *Deep learning best practices: Checkpointing your deep learning model training*, https://nusit.nus.edu.sg/services/hpc-newsletter/deep-learning-best-practices-checkpointing-deep-learning-model-training/. 2018.

[21] into deep learning, D., *Beam search*, https://d2l.ai/chapter_recurrent-modern/beam-search.html.

[22] Lewis, T., Leandro, W., Thomas, W., *Przetwarzanie języka naturalnego z wykorzystaniem transformerów. budowanie aplikacji językowych za pomocą bibliotek hugging face*. 2024.

[23] Merity, S., Xiong, C., Bradbury, J., Socher, R., *Pointer sentinel mixture models*. 2016.

[24] Merity, S., Xiong, C., Bradbury, J., Socher, R., *Pointer sentinel mixture models*. 2016.

[25] Mike, L., Yinhan, L., Naman, G., Marjan, G., Abdelrahman, M., Omer, L., Ves, S., Luke, Z., *Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension*. 2019.

[26] Mudadla, S., *Layer normalization*, https://medium.com/@sujathamudadla1213/layer-normalization-48ee115a14a4.

[27] OpenAI, *Gpt-2 large*, https://huggingface.co/openai-community/gpt2-large.

[28] OpenAI, *Gpt-2 medium*, https://huggingface.co/openai-community/gpt2-medium.

[29] OpenAI, *Gpt-2 xl*, https://huggingface.co/openai-community/gpt2-xl.

[30] Parthasarathy, V., Zafar, A., Khan, A., Shahid, A., *The ultimate guide to fine-tuning llms from basics to breakthroughs: An exhaustive review of technologies, research, best practices, applied research challenges and opportunities*.

[31] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., *Language models are unsupervised multitask learners*. 2019.

[32] Talgalili, *Z-test*, https://en.wikipedia.org/wiki/Z-test.

[33] Turing.com, *Understanding data processing techniques for llms*, https://www.turing.com/resources/understanding-data-processing-techniques-for-llms#a.-pre-processing.

[34] Yenduri, G., M, R., Selvi G, C., Y, S., Srivastava, G., Maddikunta, P.K.R., G, D.R., Jhaveri, R.H., Prabadevi, *Gpt (generative pre-trained transformer) – a comprehensive review on enabling technologies, potential applications, emerging challenges, and future directions*. 2023.

[35] Yijian, L., Aiwei, L., Dianzhi, Y., Jingjing, L., Irwin, K., *An entropy-based text watermarking detection method*, https://aclanthology.org/2024.acl-long.630.pdf.

[36] Yu, F., Deyi, X., Yue, D., *Watermarking conditional text generation for ai detection: Unveiling challenges and a semantic-aware watermark remedy*. 2024.

SPIS RYSUNKÓW

1.1 Architektura koder dekoder [15] 7
1.2 Procesu tokenizacji dla modeli GPT-3.5/GPT-4 9
1.3 Hipotetyczne przedstawienie wektorów osadzeń [12]. 10
1.4 Relacja zachodzące między wektorami [12]. 10
1.5 Przykładowa macierz $M = Q{\cdot}K$ 12
1.6 Proces maskowania. 13
1.7 Macierz przykładowych prawdopodobieństw pomiędzy wektorami zapytań i kluczy. 13
1.8 Proces tworzenia $\overrightarrow{\mathbf{E}}_i$ dla przykładowych prawdopodobieństw. 13
1.9 Wiele głów atencji [10]. 14

2.1 Beam Search [21]. 17

4.1 Architektura decoder-only [15]. 22

5.1 Przykładowa reprezentacja czerwonej i zielonej listy. 24

7.1 Wpływ parametru γ na *z-score* 34
7.2 Wpływ parametru γ na *z-score* i *perplexity* 34
7.3 Wpływ parametru δ na *z-score* 36
7.4 Wpływ parametru δ na *perplexity* i *z-score* 36
7.5 Wpływ γ i δ na otrzymywane wartości *z-score* i *perplexity* 37
7.6 Wpływ parametru γ na *z-score* 38
7.7 Wpływ parametru γ na *perplexity* i *z-score* 38

SPIS TABEL

1.1 Metody tokenizacji 9

4.1 Parametry modeli GPT-2 22

6.1 Złożoności obliczeniowe 32